

Common Tables Expressions (CTE)

Introducción a las CTE (Common Table Expressions)

Una CTE (Expresión de Tabla Común) es una forma temporal de definir una tabla derivada en una consulta SQL. También podemos decir que son una característica de SQL que permite crear una "tabla temporal" que puede ser referenciada dentro de una consulta.

A diferencia de las subconsultas, las CTE son más legibles y reutilizables. Su uso se recomienda especialmente cuando una consulta tiene múltiples partes complejas o cuando se necesita hacer referencia a un mismo conjunto de datos varias veces dentro de una consulta (CTE recursiva).

Tipos de CTE

1. CTE Simples:

- Una CTE simple se define usando la cláusula WITH seguida por una consulta SQL.
- La tabla temporal creada por la CTE solo está disponible durante la ejecución de la consulta principal que la sigue.

2. CTE Recursivas:

- Las CTE recursivas permiten hacer referencia a sí mismas, facilitando el trabajo con datos jerárquicos o con estructuras recursivas, como árboles genealógicos, estructuras organizativas, o datos que requieren cálculos iterativos.

Sintaxis básica de una CTE (simple):

```
WITH mi_CTE AS  
(  
    SELECT columnas  
    FROM tabla  
    WHERE condiciones  
)
```



Consulta principal que utiliza la CTE

```
SELECT columnas  
FROM mi_CTE;
```



- WITH: Palabra clave que introduce la CTE.
- mi_CTE: Nombre temporal de la CTE.
- La consulta después de WITH define los datos que estarán disponibles dentro de la CTE.
- La consulta principal puede usar la CTE como si fuera una tabla temporal.

Sintaxis de una CTE recursiva (solo a modo de ejemplo):

```
WITH RECURSIVE nombre_cte_rekursiva AS
(
    -- Este seria el "punto de partida" de la recursión
    SELECT columna1, columna2, ...
    FROM tabla
    WHERE condición_inicial

    UNION ALL

    -- Aquí podemos ver la recursividad (se hace referencia a la CTE)
    SELECT columna1, columna2, ...
    FROM tabla
    JOIN nombre_cte_rekursiva ON condición_rekursiva
)
```

Consulta principal que utiliza la CTE recursiva

```
SELECT columna1, columna2, ...
FROM nombre_cte_rekursiva;
```

Ventajas de las CTE:

1. Legibilidad: Hace que las consultas sean más fáciles de leer y entender, especialmente cuando se tiene que trabajar con múltiples subconsultas.
2. Reutilización: Las CTE permiten usar el mismo conjunto de datos en diferentes partes de la consulta sin tener que escribir la subconsulta repetidamente.
3. Recursividad: Las CTE también pueden ser recursivas, lo que permite resolver problemas como el manejo de jerarquías o la iteración a través de conjuntos de datos.

Ejemplos de CTE

Ejemplo 1: CTE básica para calcular promedios

```
WITH EmpleadosSalarios AS
(
    SELECT empleado_id, salario
    FROM Empleados
    WHERE salario > 3000
)

SELECT AVG(salario) AS PromedioSalario
FROM EmpleadosSalarios;
```

la CTE EmpleadosSalarios
selecciona los empleados con
un salario superior a 3000

la consulta principal calcula
el salario promedio de esos
empleados

Ejemplo 2: CTE con JOIN

```
WITH EmpleadosDepartamentos AS
(
    SELECT e.empleado_id, e.nombre, d.nombre_departamento
    FROM Empleados e
    JOIN Departamentos d ON e.departamento_id = d.departamento_id
)

SELECT *
FROM EmpleadosDepartamentos
ORDER BY nombre_departamento;
```

Ejemplo 3: CTE recursiva

```
WITH RECURSIVE jerarquia_empleados AS  
(  
    /*obtener el jefe inicial*/  
  
    SELECT id, nombre, id_jefe  
    FROM empleados  
    WHERE id_jefe IS NULL /*Comenzamos con el jefe máximo*/  
  
    UNION ALL  
  
    /*Parte recursiva: obtener los empleados supervisados por los empleados obtenidos en la  
    parte inicial*/  
  
    SELECT e.id, e.nombre, e.id_jefe  
    FROM empleados e  
    JOIN jerarquia_empleados je ON e.id_jefe = je.id  
)  
  
/*consulta ppal (llamado a la CTE recursiva)*/  
  
SELECT *  
FROM jerarquia_empleados;
```

la CTE recursiva
Jerarquía_empleados se
utiliza para construir una
jerarquía de
empleados y sus jefes

Diferencias entre SubConsultas y CTE's (tabla comparativa)

A continuación, tenemos una tabla comparativa en la que podemos observar las características principales que comparten o no, las subqueries y las cte's.

Característica	Subconsultas	CTE (Common Table Expressions)
Definición	Consultas anidadas dentro de una consulta principal	Bloques de consulta temporales referenciables
Sintaxis	Parte de las cláusulas SELECT, FROM, WHERE, etc.	Definida con WITH y puede ser recursiva
Legibilidad	Puede volverse compleja y difícil de leer	Mejora la legibilidad dividiendo consultas complejas
Reutilización	No puede ser reutilizada dentro de la misma consulta	Puede ser referenciada múltiples veces en una consulta
Recursividad	No admite recursividad	Admite recursividad (útil para datos jerárquicos)
Compatibilidad	Compatibles con casi todas las versiones de SQL	Requiere versiones posteriores a MySQL 8.0

Ejemplo (para comparar)

1. Subconsulta para Obtener el Cliente con más facturación

```
SELECT cliente_id, nombre
FROM clientes
WHERE cliente_id = (SELECT cliente_id
                    FROM ventas
                    GROUP BY cliente_id
                    ORDER BY SUM(total) DESC
                    LIMIT 1);
```

2. CTE para Obtener el Cliente con más facturación

```
WITH ventas_totales AS
(
    SELECT cliente_id, SUM(total) AS total_ventas
    FROM ventas
    GROUP BY cliente_id
)

SELECT c.cliente_id, c.nombre
FROM clientes c
JOIN ventas_totales v ON c.cliente_id = v.cliente_id
ORDER BY v.total_ventas DESC
LIMIT 1;
```

Entonces, por lo visto hasta aquí, ¿cuándo nos conviene usar una SubQuery y cuando una CTE?

CTE cuando:

- Queremos mejorar la legibilidad y estructura del código.
- Necesitamos reutilizar resultados intermedios.
- Trabajamos con datos jerárquicos o recursivos.
- Deseamos separar la lógica de negocio en componentes claros.
- Utilizamos versiones modernas de MySQL (8.0 o superior).

Subconsultas cuando:

- La consulta sea simple y no necesite ser reutilizada en múltiples lugares.
- No tengamos que trabajar con datos recursivos.
- La compatibilidad con versiones más antiguas de MySQL sea un factor.